Luke Colehower
Applied Database Technologies
Lab 3: Table JOINs
5 March 2024

Chapter 4: How to retrieve data from two or more tables

1. Write a SELECT statement that joins the categories table to the products table and returns these columns: product_id, catgeory_id, category_name, product_name, list_price.

   Sort the result set by the category_name column and then by the product_name column in ascending sequence.

   SELECT categories.category_name, categories.category_id,
   products.product_id, products.product_name,
   products.list_price, products.category_id
   FROM categories
   JOIN products ON categories.category_id = products.category_id
   ORDER BY category_name AND product_name;

2. Write a SELECT statement that joins the Customers table to the Addresses table and returns these columns: first_name and last_name separated by a space as one column using CONCAT() (i.e., **John Doe)**, line1, line2, city, state, zip_code.

   Return one row for each address for the customer with an email address of allan.sherwood@yahoo.com.

   SELECT c.customer_id, CONCAT(c.first_name, ' ' , c.last_name) AS "Customer Name", a.line1,

   a.line2, a.city, a.state, a.zip_code, a.customer_id

   FROM customers AS c

   JOIN addresses AS a ON c.customer_id = a.customer_id

   WHERE email_address = 'allan.sherwood@yahoo.com';

3. Write a SELECT statement that joins the customers table to the addresses table and returns these columns: first_name and last_name separated by a space as one column using CONCAT() (i.e., **John Doe)**, line1, line2, city, state, zip_code.

   Return one row for each customer, but only return addresses that are the billing address for a customer

SELECT c.customer_id, CONCAT(c.first_name, ' ' , c.last_name) AS "Customer Name", a.line1,

a.line2, a.city, a.state, a.zip_code

FROM customers AS c

JOIN addresses AS a ON c.billing_address_id = a.address_id

ORDER BY c.customer_id;

4. Write a SELECT statement that joins the customers, orders, order_Items, and products tables. This statement should return these columns: first_name, last_name, order_id, order_date, product_name, item_id, item_price, discount_amount, and quantity.

   Use aliases for the tables.

   Sort the final result set by the last_name, order_date, and product_name columns in descending order.

   SELECT c.customer_id, CONCAT(c.first_name, ' ' , c.last_name) AS 'Customer Name' , o.order_id, o.customer_id,

   o.order_date, t.product_id, t.item_id, t.item_price, t.discount_amount, t.order_id, t.quantity,

   p.product_id, p.product_name, p.list_price, p.date_added

   FROM customers AS c

   JOIN orders AS o ON c.customer_id = o.customer_id

   JOIN order_items AS t ON o.order_id = t.order_id

   JOIN products AS p ON p.product_id = t.product_id

   ORDER BY c.last_name DESC, order_date DESC, product_name DESC;

5. Write a SELECT statement that returns the product_name and list_price columns from the products table.

   Return one row for each product that has the same list price as another product. Hint: Use a self-join (products to products) with a condition to check that the product_id columns are **NOT** equal and the list_price columns are equal.

   Sort the result set by the product_name column.

   SELECT p.product_id, p.product_name, p.list_price

FROM products AS p

JOIN products p2 ON p.product_name = p2.product_name AND p.list_price = p2.list_price

WHERE p.product_id != 0 AND p.list_price = p2.list_price

ORDER BY p.product_name;

6. Write a SELECT statement that returns these two columns:
    - category_name: The category_name column from the categories table
    - product_id: The product_id column from the products table

    Return one row for each category that has never been used. Hint: Use an outer join and only return rows where the product_id column contains a null value.

    SELECT c.category_id, c.category_name, p.product_id

    FROM categories AS c

    LEFT JOIN products p ON c.category_id = p.category_id

    WHERE p.product_id IS NULL;

7. Use the UNION operator to generate a result set consisting of three columns from the orders table:
    - ship_status: A calculated column that contains a value of SHIPPED or NOT SHIPPED
    - order_id: The order_id column
    - order_date: The order_date column

    If the order has a value (NOT NULL) in the ship_date column, the ship_status column should contain a value of SHIPPED. Otherwise, it should contain a value of NOT SHIPPED.

    Sort the final result set by the order_date column descending.

SELECT order_id, order_date, ship_date, "SHIPPED" AS SHIPMENT

FROM orders

WHERE ship_date IS NOT NULL

UNION

SELECT order_id, order_date, ship_date, "NOT SHIPPED"

FROM orders

```
WHERE ship_date IS NULL;
```